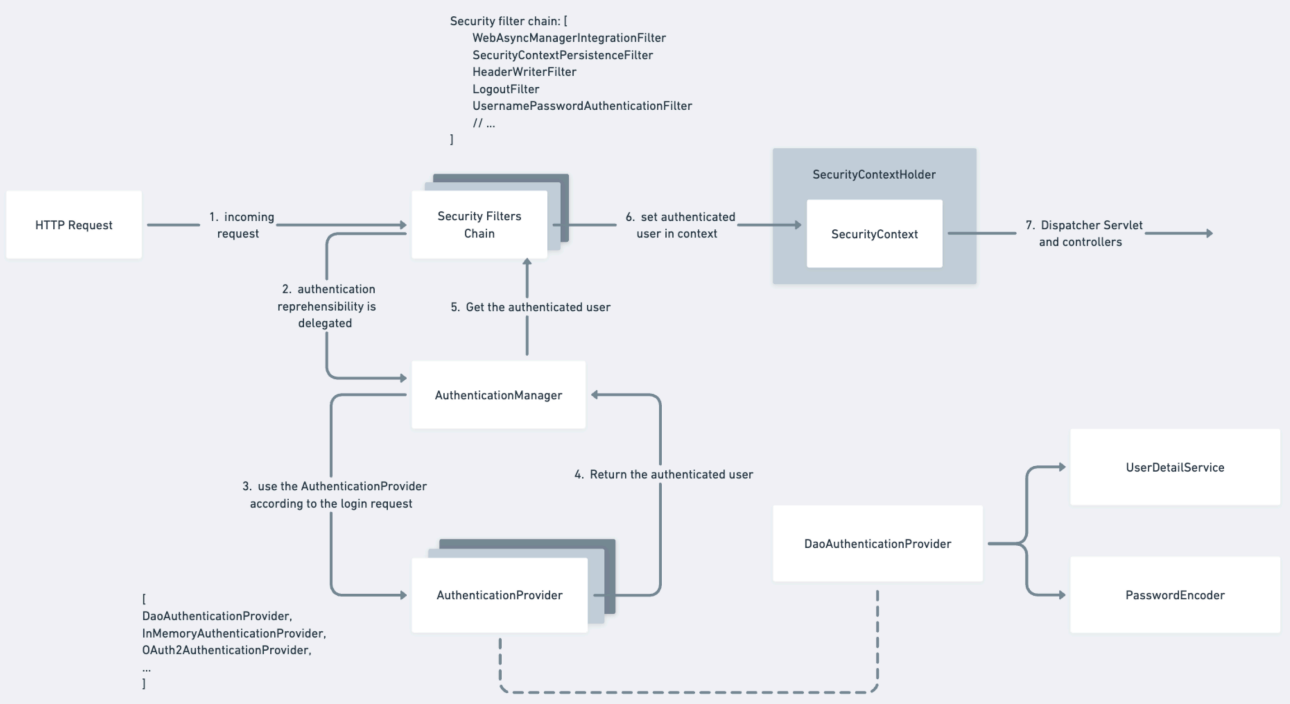
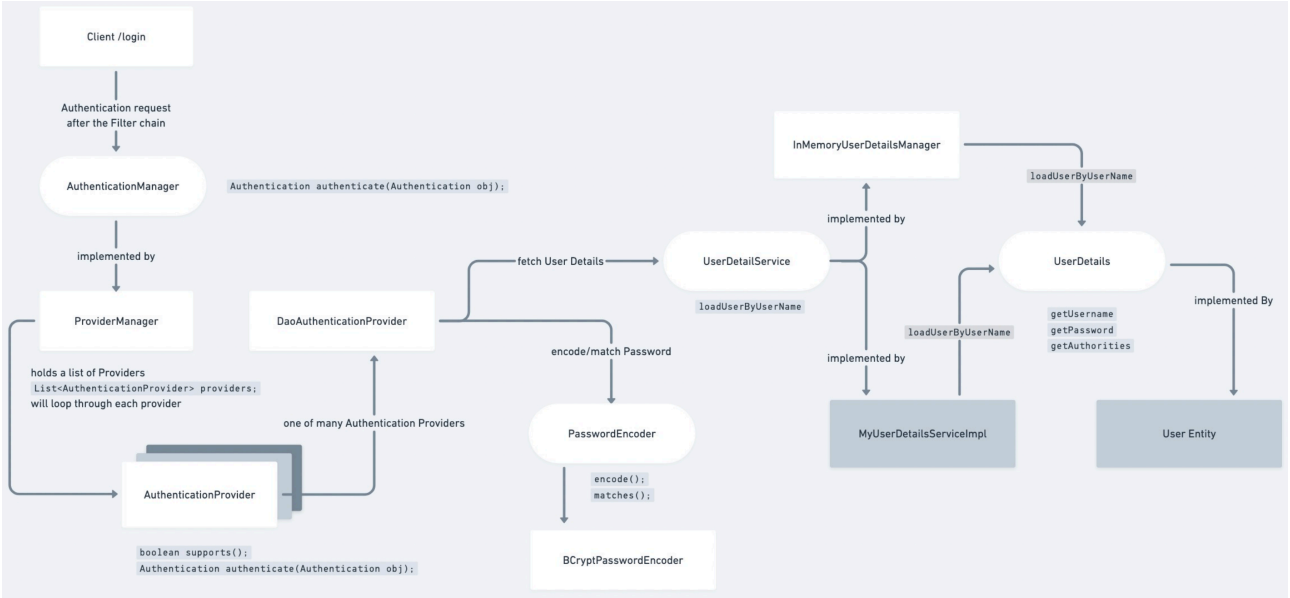
5.3 Internal Working of Spring Security, Part - 2
Core Components

• Internal Authentication Flow of Spring Security :

- When a user attempts to log in, Spring Security processes the request through a **series of filters** and **authentication components** before allowing access to the requested resource.
- The complete authentication flow is illustrated below.



- **Zoomed version:**



5.3 Internal Working of Spring Security, Part - 2
Core Components

- **Explanation:**
  - **Step 1:** Client Sends Authentication Request
    - The process begins when a client submits a login request (e.g. POST / login) with credentials: {"Username" : "sibasundar", "Password" : "password123"}
    - The request first reaches the **Spring Security Filter Chain**, before reaching the controller.
  - **Step 2:** Security Filter Chain Intercepts the Request
    - Every incoming request passes through the **Security Filter Chain**, which consists of several security filters such as:
      - SecurityContextPersistenceFilter
      - CsrfFilter
      - LogoutFilter
      - UsernamePasswordAuthenticationFilter
      - AuthorizationFilter
    - For a username/password login request, the important filter is: UsernamePasswordAuthenticationFilter.
    - This filter extracts the **username** and **password** then creates an **unauthenticated UsernamePasswordAuthenticationToken** and delegates authentication to the **AuthenticationManager**.
  - **Step 3:** AuthenticationManager Handles Authentication
    - **AuthenticationManager** is the central authentication interface in Spring Security having a method called **authenticate** which takes an authentication object as a parameter.
    - It does not authenticate users directly. Instead, it delegates the authentication request to one of the configured AuthenticationProviders. In Spring Security, the default implementation is: ProviderManager
  - **Step 4:** ProviderManager Selects an AuthenticationProvider
    - **ProviderManager** maintains a list of registered **authenticationProviders** and a **supports** method, It iterates through the list and calls: **supports**(**authentication**) to determine which provider can authenticate the given request.
    - For a username/password login, it selects: **DaoAuthenticationProvider**
  - **Step 5:** DaoAuthenticationProvider Retrieves User Details
    - **DaoAuthenticationProvider** authenticates users against application data and delegates user retrieval to: **UserDetailsService** by invoking, loadUserByUsername.
    - `UserDetails user = userDetailsService.loadUserByUsername("sibasundar");`
  - **Step 6:** UserDetailsService Loads the User
    - **UserDetailsService** is an interface responsible for locating user information.
    - Spring Security provides several implementations, like **InMemoryUserDetailsManager** or developers can create a custom implementation, ex: **MyUserDetailsServiceImpl**.
    - The implementation typically queries the database, and calls method like: `UserEntity user = userRepository.findByEmail(username);`

5.3 Internal Working of Spring Security, Part - 2
Core Components

- **Step 7:** UserDetails Object is Returned
  - The user's information is converted into a UserDetails object.
  - UserDetails contains:
    - Username
    - Encoded Password
    - Authorities (Roles/Permissions)
    - Account Status
- **Step 8:** Password Verification
  - After obtaining the user details, **DaoAuthenticationProvider** verifies the password and delegates password comparison to the configured **PasswordEncoder**.
  - PasswordEncoder has a **match** method which takes a raw and an encoded password return true is they matches with the encoded password stored in DB otherwise returns false.
  - If **matches( )** returns true then authentication **succeeds** otherwise throws **BadCredentialsException**.
- **Step 9:** Authentication Object is Returned
  - Upon successful **authentication**, **DaoAuthenticationProvider** creates an authenticated **Authentication** object and returned it to **AuthenticationManager**.

Authentication object:	Authenticated = true Principal = UserDetails Authorities = ROLE_ADMIN, ROLE_USER

- **Step 10:** SecurityContext is Updated
  - The Security Filter Chain stores the authenticated user inside the **SecurityContext**.
  - **SecurityContextHolder** uses a **ThreadLocal** by default, making the authentication information available throughout the processing of the current request.
- **Step 11:** Request Continues to the Controller
  - Since authentication is successful, Spring Security forwards the request to the **DispatcherServlet**. Then normal Spring MVC flow will continues:
    - DispatcherServlet —> Controller —> Service —> Repository
  - Controllers can access the authenticated user using: Authentication authentication or @AuthenticationPrincipal .

5.3 Internal Working of Spring Security, Part - 2
Core Components

- **Component Responsibilities :**

Component	Responsibility
Security Filter Chain	Intercepts every HTTP request and delegates authentication.
UsernamePasswordAuthenticationFilter	Extracts username / password and creates an Authentication object.
AuthenticationManager	Coordinates the authentication process.
ProviderManager	Chooses the appropriate AuthenticationProvider.
AuthenticationProvider	Performs the actual authentication logic.
DaoAuthenticationProvider	Authenticates users using a UserDetailsService and PasswordEncoder.
UserDetailsService	Loads user information from a data source.
UserDetails	Represents the authenticated user's credentials and authorities.
PasswordEncoder	Verifies the raw password against the stored encoded password.
SecurityContextHolder	Stores the authenticated user for the current request.